GovExam Alert: A Government Exam Notification Portal

Project Proposal for Software Engineering – UCS503L

Submitted to: Dr. Paramveer Sidhu
Thapar Institute of Engineering and Technology
Computer Science and Engineering Department

Bir Angad Singh* Raghav Mittal† Aryan Juneja‡

14 August 2026

1 Higher-order goal . . . secondary framing

This project aims to improve access to public-sector opportunities by reducing the time and effort citizens spend searching for government job and exam notifications scattered across dozens of official websites. While the broader goal is equitable access to public employment information, the immediate engineering objective is to translate this into a measurable, deployable web application that aggregates, organizes, and alerts users about relevant exam notifications.

2 Time-to-value . . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core browse-and-search workflow first and validating it with a small group of student users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations and targeting CI-backed automation early, to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration (a working notification listing and search feature), then improved based on user feedback.

3 Problem Statement

Millions of students and job seekers in India track government exam notifications (e.g., SSC, UPSC, State PSCs, Railways, Banking) that are published on separate, often poorly designed departmental websites. Common pain points include:

- **Fragmentation:** notifications are spread across many official portals, PDFs, and newspaper ads, with no single reliable source.
- **Missed deadlines:** candidates frequently miss application windows or admit-card release dates due to lack of timely reminders.
- **Poor discoverability:** existing government sites are hard to search or filter by qualification, category, or state.

*Roll No: 1024031150
†Roll No: 1024030747
‡Roll No: 1024030731

- **Information overload:** unofficial aggregator sites mix outdated, duplicate, or unverified postings with genuine ones, causing confusion.

Why this matters now (contextual relevance): With a growing number of students preparing for competitive government exams, even a simple, reliable, and well-organized notification portal can noticeably reduce missed opportunities and search effort.

4 Proposed Solution . . . Software Proposition

4.1 Overview

Build a web-based "GovExam Alert" platform with the following components, matching the high-fidelity prototype designed in Figma:

- **Home dashboard:** a searchable exam feed with quick-stat cards at the top (Total Exams, Active, Urgent, Reminders) giving users an at-a-glance summary.
- **Exam cards:** each notification is shown as a card with an organization tag and an urgency badge (e.g., *Ending Soon*, *Urgent*, *New*), post count, last date to apply, exam date, and a live "days left" countdown.
- **Save and Apply actions:** each card has a *Save* button (bookmark for later) and a direct *Apply* button linking to the official source.
- **Reminders tab:** a dedicated bottom-navigation tab where users review notifications they have bookmarked or set alerts for.
- **Settings:** notification preferences (push notifications, deadline reminders sent a fixed number of days before the last date, and optional email notifications), along with theme and language preferences (see Settings screen details below).
- **Admin panel (staff-facing, not shown in the citizen prototype):** a small team of student-admins can add, edit, and verify notifications sourced from official sites before they appear on the dashboard.

4.2 Core workflow

1. Admin adds a notification with structured fields (title, organization, category, post count, last date, exam date, official link) via the admin panel.
2. System validates required fields, auto-computes the urgency badge (*New*/*Ending Soon*/*Urgent*) from the days-left value, and publishes the notification to the public dashboard.
3. Citizen/student searches or scans the dashboard, and taps *Save* on notifications of interest, or *Apply* to go directly to the official source.
4. System sends a reminder alert (push and/or email, per the user's Settings) a configurable number of days before the application deadline.
5. User reviews all saved/reminded exams under the dedicated *Reminders* tab.

4.3 UI/UX prototype reference

A high-fidelity prototype was designed in Figma covering both Android (Material Design 3) and iOS (Human Interface Guidelines) variants, so that the responsive web build can borrow consistent, platform-appropriate visual patterns (card-based layout, bottom navigation, floating action elements). Key screens are summarized in Table .

Screen	Key elements
Home dashboard	Search bar; quick-stat cards (Total, Active, Urgent, Reminders); exam cards with organization tag, urgency badge, post count, last date, exam date, days-left countdown, Save and Apply buttons; bottom navigation (Home, Reminders, Settings).
Settings	Guest/user profile; notification toggles (Push, Deadline Reminders – 3 days before, Email); theme and language preferences; privacy policy and help/support links.
Design system cover	Confirms dual design-system coverage (Material Design 3 for Android, Human Interface Guidelines for iOS) used as the visual reference for the responsive web build.

Table 1: Prototype screens and their key UI elements

4.4 Operational constraints for an educational, second-year setting

- Keep the solution usable on low-to-mid range devices via responsive web design, since many users access it on mobile.
- Avoid dependence on advanced research algorithms (e.g., no ML ranking in the first iteration); focus on correct workflow, search, and reminders.
- Design for deployability using common free-tier web hosting and managed databases, appropriate for a three-person, third-year team.

5 Solution Approach . . . Engineering focus

This is an introductory software engineering course project, so we focus on core web-application correctness, usability, and a repeatable delivery pipeline rather than research novelty.

5.1 Search and filtering . . . non-research

Search will be implemented with simple, well-understood techniques:

- Structured filters on organization, category, state, and qualification (standard database queries, not free-text ranking).
- A basic keyword search over title/organization fields using SQL `LIKE`/full-text search, without claiming state-of-the-art relevance ranking.
- Sorting by nearest deadline as the default view, since urgency matters most to users.

5.2 Web architecture . . . web-based solution

A typical 3-tier web architecture:

- **Frontend:** responsive UI for browsing, searching, and bookmarking notifications (built with React/HTML-CSS-JS).
- **Backend API:** authentication, CRUD for notifications, bookmarks, and reminder scheduling logic (built with Node.js/Express or Flask).
- **Database:** stores users, notifications, bookmarks, and reminder logs (e.g., PostgreSQL/MySQL/MongoDB).

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to a staging environment.
- Enable safe and frequent releases of incremental features, e.g., adding the alerts module after the core listing is stable.

6 Evaluation Criterion . . . measurable and attributable

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Time-to-discovery (TTD): time taken by a test user to find a relevant notification matching a given profile (e.g., "state government, graduate, clerical post") using the portal, compared to manually searching official sites.

- **Target:** median TTD ≤ 60 seconds during pilot testing, at least 3x faster than manual search.
- **Attribution:** measured via timed usability tasks with pilot users and application logs (search-to-click timestamps).

6.2 Secondary evaluation metrics

Metric	Description	Target
Alert reliability	Percentage of scheduled reminders successfully delivered before the relevant deadline	$\geq 95\%$
Data freshness	Average delay between an official notification being published and it appearing on the portal	≤ 24 hours
User clarity	User-reported clarity of notification details using a simple 1–5 feedback question	Average $\geq 4/5$
Reliability	Availability of staging/production for browsing and login	$\geq 99\%$ uptime in pilot

Table 2: Secondary evaluation metrics

6.3 Pilot validation plan . . . high-level

- Recruit 10–15 pilot users (fellow students preparing for government exams) and 1–2 student-admins to simulate the verification workflow.
- Compare time-to-discovery and satisfaction before/after within the iteration window using short interviews and task timing.

7 Scalability . . . with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in the number of notifications, categories, and concurrent users by:

- decoupling frontend and backend,
- enabling pagination and indexing for common search queries,
- using a stateless API design so the backend can be horizontally scaled.

2. **Operationally deployable (practically):** the system should run on common web hosting:
   - managed database (e.g., a free-tier cloud database),
   - containerized or platform-hosted deployment (e.g., Render, Railway, Vercel),
   - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project, appropriate for a second-year skill level:

- Web framework for REST APIs and reactive frontend pages (Express/Flask + React).
- Managed relational or document database (PostgreSQL/MongoDB Atlas free tier).
- Authentication approach (token-based, e.g., JWT, or session-based).
- Scheduled job/cron mechanism for sending reminder alerts.
- Test framework for unit/integration tests (e.g., Jest/PyTest).
- CI runner and staging deployment target (GitHub Actions + free-tier host).

We will use these mature, well-documented components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables . . . iteration-friendly

Iteration	Deliverable	Priority
9.1 Initial (first iteration)	Home dashboard: search bar + quick-stat cards (Total, Active, Urgent, Reminders)	Must-have
	Exam cards with organization tag, urgency badge, post count, dates, days-left countdown	Must-have
	Save (bookmark) and Apply (official link) actions on each card	Must-have
	Admin panel: add/edit/publish a notification	Must-have
	Basic logging and timestamps for TTD measurement	Must-have
	CI: build + backend unit tests + linting	Must-have
	CD to staging with a single command or pipeline job	Must-have
9.2 Subsequent	Reminders tab + Settings (push, deadline, and email notification toggles)	Should-have
	User accounts and category preferences	Should-have
	Keyword search with basic full-text matching	Could-have
	Admin dashboard for notification counts and freshness metrics	Could-have

Table 3: Project deliverables by iteration

10 Risks and mitigations

Risk	Mitigation
Data accuracy	Incorrect or outdated notifications could mislead users; mitigate by always linking to the official source and having a student-admin verification step before publishing.
Data entry overhead	Manually adding notifications is time-consuming for a three-person team; mitigate by keeping the admin form minimal and scoping the pilot to a small set of exam categories.
Evaluation measurement ambiguity	Instrument timestamps and define clear notification-status states (Draft, Published, Expired) before development begins.
Operational issues in pilot	Use staging early; ensure CI/CD produces repeatable deployments before the pilot begins.

Table 4: Risks and mitigations

11 Summary

This project proposes a deployable web platform, GovExam Alert, to reduce the time and effort citizens spend discovering and tracking government exam and job notifications. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-discovery). As a web-based application project scoped for a third-year, three-member team, it remains focused on engineering workflow, usability, and scalability readiness rather than research-driven novelty.